Evaluación Parcial N°2 — Encargo: Desarrollo de un Agente Funcional

Ingeniería de Soluciones con IA (ISY0101) — Agente informativo BAES para Pluxee Chile
Integrante: Ignacio Gómez — RUT 20.248.694-0
Profesor: Francisco Macaya
Repositorio: https://github.com/Lucem-nacho/pluxee-baes-agent

1. Diseño e implementación del agente (IE1, IE2)

En EP1 el agente era una secuencia fija de funciones Python (clasificar → recuperar → comercios → arbitrar → generar). En EP2 se reemplazó el tramo INFORMATIVA por un AgentExecutor de LangChain (langchain-classic, sobre langchain-google-genai + Gemini), donde el propio LLM decide qué herramienta invocar, en qué orden y cuándo detenerse.

Cuatro tools expuestas al agente (agent/tools.py):

- **buscar_normativa** — envuelve el pipeline RAG de EP1 (ChromaDB + arbitraje por recencia), sin reescribirlo.
- **buscar_comercio** — envuelve la consulta SQL estructurada sobre comercios, también heredada de EP1.
- **consultar_historial** — nueva: lee la memoria de largo plazo (tabla interacciones) del id_usuario de la sesión.
- **escalar_a_ejecutivo** — nueva, de escritura: el agente puede derivar a un ejecutivo a mitad de su propio razonamiento, no solo el clasificador de entrada.

agent/classifier.py se mantiene sin cambios de lógica como filtro de entrada barato: evita instanciar un ciclo completo de tool-calling (varias llamadas al LLM) en consultas claramente fuera de alcance, antes de llegar al agente.

Validado con llamadas reales a Gemini (no solo diseño teórico): los 4 escenarios de scripts/check_agent_executor.py confirmados end-to-end — detalle de qué se encontró y corrigió en la sección 5.

Elección de framework: LangChain fue elegido principalmente por su alta flexibilidad y su ecosistema maduro para la integración fluida con la API de Gemini a través de langchain-google-genai. A diferencia de Autogen, que está diseñado fundamentalmente para sistemas multi-agente complejos, y Haystack, que destaca en pipelines de búsqueda semántica (RAG) pero puede ser más rígido para la toma de decisiones dinámicas, LangChain (con su AgentExecutor) ofrece un equilibrio ideal para este caso de uso. Permite escalar fácilmente la adición de nuevas herramientas (como buscar_normativa, buscar_comercio, consultar_historial y escalar_a_ejecutivo) y facilita que el LLM decida autónomamente la ruta óptima de ejecución. El principal trade-off técnico evaluado es la alta volatilidad de sus versiones (lo que obligó a migrar a langchain_classic durante la integración), pero su capacidad nativa para manejar "Tool Calling" de forma adaptativa justifica plenamente su elección frente a alternativas más estructuradas.

2. Memoria y recuperación de contexto (IE3, IE4)

Se implementaron dos mecanismos de memoria genuinamente distintos — no una sola tabla con dos nombres — porque la pauta pide corto y largo plazo como cosas separadas:

	Corto plazo	Largo plazo
Dónde vive	st.session_state.mensajes (navegador)	Tabla interacciones en SQLite (disco)
Sobrevive a	Nada — se pierde al cerrar la pestaña	Cerrar la app, reiniciar el proceso
Quién la usa	El prompt del clasificador (historial_contexto)	La tool consultar_historial, que el LLM decide invocar
Identificada por	La sesión del navegador (implícito)	id_usuario, escrito a mano por el estudiante

Recuperación de contexto semántico (IE4) ocurre en dos niveles: buscar_normativa reutiliza la recuperación vectorial de EP1 (embeddings sentence-transformers + arbitraje por recencia) para el fragmento normativo relevante; consultar_historial recupera interacciones pasadas por id_usuario para dar continuidad entre sesiones distintas, no solo dentro de una conversación.

Hallazgo real: el prompt del clasificador heredado de EP1 enviaba preguntas de memoria ("¿Qué te pregunté la última vez?") a ESCALAR por considerarlas "ambiguas", dejando consultar_historial inalcanzable en la práctica pese a estar implementada y probada. Se amplió prompts/clasificador.py con una aclaración mínima y un ejemplo, sin tocar el resto del prompt.

Solución de memoria: Se optó por SQLite para la memoria de largo plazo debido a que proporciona persistencia de datos en disco de manera local y liviana, asegurando que las interacciones del usuario sobrevivan al cierre de la aplicación o al reinicio del proceso. Se descartó el uso de Redis porque añadiría complejidad innecesaria a la infraestructura para el volumen actual del proyecto, y se descartó un Vector Store con metadata porque la recuperación del historial requiere coincidencia exacta por el

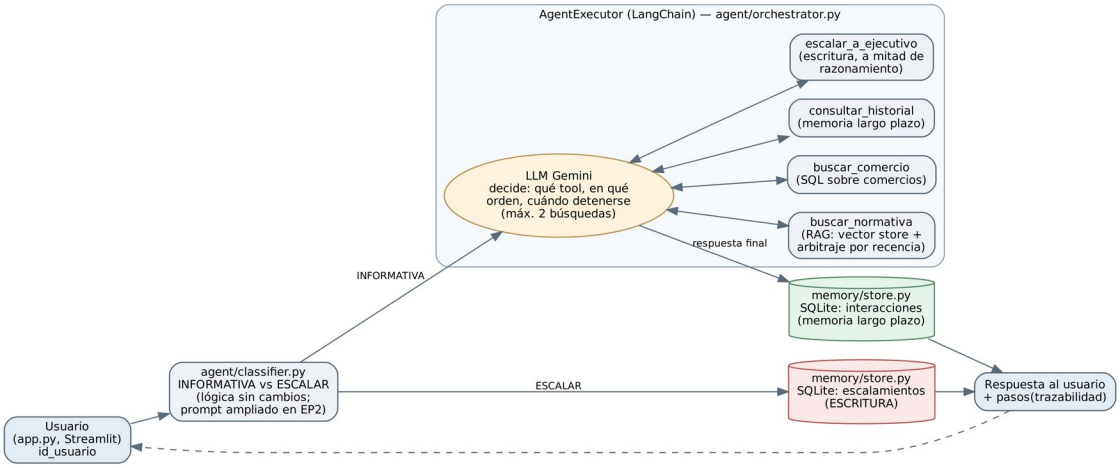
identificador del usuario, no similitud semántica. La principal limitación técnica de esta elección, y de la arquitectura actual, es la ausencia de un sistema de autenticación real. Al ingresar el id_usuario manualmente sin validación de credenciales, existe un riesgo de privacidad donde cualquier persona que digite un ID existente podría acceder al historial de consultas de otro estudiante.

3. Planificación y toma de decisiones adaptativas (IE5)

El mismo agente llega a caminos distintos según lo que encuentra, no por una ruta decidida de antemano:

- Consulta normativa general → solo dispara buscar_normativa.
- Consulta sobre un comercio puntual → dispara buscar_comercio y buscar_normativa, cruzando la restricción particular del comercio con la regla general (corregido tras detectar que el agente ignoraba el comercio — hallazgo #5, sección 5).
- Consulta que resulta ambigua o contradictoria al investigar → puede terminar en escalar_a_ejecutivo, una decisión que en EP1 solo podía tomar el clasificador, antes de ver ningún resultado de búsqueda.

Planificación acotada: regla de máximo 2 búsquedas antes de concluir honestamente "no encontré información" en vez de reformular indefinidamente (hallazgo #3, sección 5) — balance entre explorar y la cuota limitada del free tier de Gemini (20 solicitudes/día).



Orquestación de componentes del agente — clasificador, AgentExecutor con 4 tools, y los dos caminos de escritura en SQLite.

agent/orchestrator.py expone resultado["pasos"] como traza de las decisiones tomadas; app.py lo muestra junto a la respuesta, pensado para la demo en vivo de la presentación (IE7).

4. Documentación técnica (IE6)

README.md documenta instalación, construcción del índice, ejecución y verificación del agente. docs/arquitectura.md registra cada decisión de diseño con evidencia medida —no solo narrativa— para EP1 y EP2 (capturas de errores reales, valores de similitud, resultados de escenarios). El diagrama de la sección 3 resume la orquestación completa de componentes.

Cobertura de pruebas: 19 pruebas automatizadas (10 en tests/test_memory_store.py, 6 en tests/test_orchestrator.py con un modelo LLM falso para no gastar cuota real, y 3 en tests/test_build_vectorstore.py) más los 4 escenarios end-to-end verificados con Gemini real (scripts/check_agent_executor.py).

5. Hallazgos de la primera ejecución real con LangChain

agent/tools.py y agent/orchestrator.py se escribieron sin poder instalar LangChain en el entorno de desarrollo original. Su primera ejecución real (2026-10-08, LangChain 1.4.0 + langchain-google-genai 4.4.0 + Gemini) encontró 5 problemas que ninguna prueba sin red podía ver, todos corregidos:

#	Problema	Causa	Corrección
1	Import roto	AgentExecutor ya no existe en langchain.agents en LangChain 1.x	Import movido a langchain_classic; paquete agregado a requirements.txt
2	SQLite rechazaba el resultado	El contenido final del agente podía llegar como lista de bloques, no str	Normalización a texto en agent/orchestrator.py

#	Problema	Causa	Corrección
3	Bucle de búsquedas	Ante una consulta fuera de cobertura, el agente reformulaba la búsqueda hasta agotar las 6 iteraciones máximas	Regla de máx. 2 búsquedas + mensaje honesto de "no encontré información" en prompts/agente.py
4	Memoria inalcanzable	El clasificador (prompt de EP1) marcaba preguntas de memoria como ESCALAR	Ampliación mínima de prompts/clasificador.py
5	Comercio ignorado	El agente respondía solo con la norma general aunque se nombrara un comercio específico	Regla nueva en prompts/agente.py: cruzar buscar_comercio y buscar_normativa

Verificación con Gemini real: los 4 escenarios confirmados end-to-end. normativa, comercio y escalamiento en la primera ejecución (2026-10-08); memoria quedó parcial ese día por agotamiento de la cuota diaria y se confirmó el 2026-10-21: el clasificador dejó pasar "¿Qué te pregunté la última vez?" como INFORMATIVA, el agente llamó solo a consultar_historial y recuperó las dos consultas anteriores del mismo id_usuario. En el escenario comercio, el agente llamó a buscar_normativa y a buscar_comercio y cruzó ambas fuentes; en escalamiento, el clasificador derivó antes de llegar al agente y la derivación quedó registrada en SQLite.

Riesgo no resuelto: un 429 o 503 persistente en la ruta del agente (observado durante la verificación) propaga la excepción y app.py mostraría un traceback al estudiante. Documentado como mejora futura en docs/arquitectura.md.

Interpretación de los hallazgos: Los cinco problemas detectados durante la integración revelan una brecha significativa entre un diseño teórico (o con pruebas unitarias offline) y la ejecución real con modelos generativos. Errores como el bucle infinito de búsquedas (hallazgo 3) o la omisión del contexto del comercio (hallazgo 5) demuestran que los LLMs pueden comportarse de forma impredecible y divergente en producción si no se imponen "guardarraíles" estrictos (como limitar a máximo 2 búsquedas) e instrucciones explícitas en los prompts. Por otro lado, los fallos de formato y dependencias (hallazgos 1 y 2) evidencian que el ecosistema de IA cambia con demasiada rapidez. Para una próxima vez, el enfoque cambiaría hacia realizar pruebas de integración end-to-end con llamadas reales a la API en etapas mucho más tempranas del desarrollo, en lugar de confiar en que el éxito de las pruebas locales garantizará el comportamiento del LLM.

6. Limitaciones conocidas y trabajo futuro

- id_usuario no es autenticación real: cualquiera que escriba el mismo valor accede a ese historial.
- Dos mecanismos de reintento distintos para el mismo LLM (llm/client.py del clasificador vs. ChatGoogleGenerativeAI del AgentExecutor) sin unificar bajo un solo wrapper.
- Errores de cuota/servidor (429/503) no se manejan en la ruta del agente — ver riesgo no resuelto, sección 5.
- Cobertura del corpus: la regla de sellos "Alto en" (Ley N° 20.606) no estaba en el manual; se detectó al preguntar por bebidas energéticas (el agente derivó en vez de inventar) y se agregó a la FAQ desde ChileAtiende. La recuperación de esa regla depende de la redacción de la consulta (ver docs/arquitectura.md).
- Dos hallazgos posteriores a la verificación, ya corregidos: (a) un tweet simulado plantado en EP1 para probar el arbitraje contradecía la regla real (el saldo es solo para alimentos) y hacía que el agente aceptara la compra de fármacos; se retiró del corpus vivo. La regla de recencia no distingue autoridad de la fuente, así que una publicación informal puede pisar al manual oficial. (b) La reconstrucción del índice usaba upsert y conservaba fragmentos obsoletos (180 en vez de 177); ahora recrea la colección en cada corrida, con pruebas de regresión.

7. Conclusiones

El desarrollo de la Evaluación Parcial 2 logró cumplir exitosamente con los requerimientos planteados, consolidando un agente funcional capaz de integrar herramientas de búsqueda, escritura y recuperación de memoria mediante un esquema de planificación adaptativa. Las mayores dificultades reales no estuvieron en la lógica del código, sino en gobernar el comportamiento del LLM en tiempo de ejecución (evitando bucles y respuestas genéricas) y en lidiar con la inestabilidad de las librerías subyacentes. Como trabajo futuro, además de implementar un sistema de autenticación real y unificar los wrappers de conexión a la API, es imperativo desarrollar un manejo de excepciones robusto en la ruta del agente para gestionar caídas del servidor o bloqueos por cuota (errores 429/503) de forma transparente, evitando que el usuario final reciba errores técnicos en la interfaz.

8. Reflexión individual

Ignacio Gómez: En lo personal, este proyecto representó un gran desafío al pasar de un paradigma de programación lineal a uno probabilístico, donde el flujo de control se delega a un modelo de IA. Mi contribución principal se enfocó en la implementación de la orquestación mediante LangChain, la integración de las herramientas y la configuración de los mecanismos de memoria a corto y largo plazo. Mi mayor dificultad personal fue enfrentar la transición de un diseño teórico funcional a la práctica: fue frustrante ver cómo el código que pasaba las pruebas locales fallaba en vivo debido a respuestas mal formateadas del modelo o al rápido agotamiento de la cuota diaria de Gemini. Este proceso me dejó como gran aprendizaje la importancia del diseño de sistemas resilientes, entendiendo que al trabajar con IA se deben anticipar constantemente las "alucinaciones" del modelo y las restricciones de la infraestructura externa.

9. Declaración de uso de Inteligencia Artificial

Se usó Claude (Anthropic), incluyendo Claude Code, como apoyo para: diseñar junto con el estudiante la arquitectura técnica del AgentExecutor y sus tools, generar el diagrama de orquestación de la sección 3, redactar la documentación técnica (README, docs/arquitectura.md, y las secciones descriptivas de este informe salvo conclusiones y reflexión individual), depurar errores reales de integración con LangChain/Gemini (import roto, bucle de iteraciones, normalización de salida, listados en la sección 5), y buscar y contrastar fuentes públicas sobre la regla de sellos para redactar la entrada de FAQ correspondiente. Las decisiones de diseño, la interpretación de los hallazgos, las conclusiones y la reflexión individual son de autoría propia del estudiante, redactadas sin apoyo de IA conforme a esta pauta. Referencia de prompting: docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview.

Referencias (APA)

Junta Nacional de Auxilio Escolar y Becas. (2026). Resolución Exenta N° DN-00556/2026: Aprueba modificaciones al Manual de Orientaciones Técnicas para la asignación de la Beca de Alimentación para la Educación Superior (BAES) 2026, y fija su texto refundido. JUNAEB.

ChileAtiende. (2026, 8 de enero). Beca de Alimentación para la Educación Superior (BAES) [Ficha de trámite]. Gobierno de Chile. https://www.chileatiende.gob.cl/fichas/2125/1/pdf

LangChain. (2026). AgentExecutor and legacy agent APIs — langchain-classic documentation. https://python.langchain.com/

Google. (2026). Gemini API documentation — Google AI for Developers. https://ai.google.dev/gemini-api/docs

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. https://arxiv.org/abs/1908.10084